

pArc

Process for Agentic oRchestration of symmetriC Engineering

An Architecture-Centric Position Paper for Role-Orchestrated, Vendor-Neutral Agentic Engineering

Draft v0.0.0 — feasibility and structure review, not a validated standard

10 September 2026

Abstract

Agentic software development is moving from single-session coding assistance toward systems in which planning, architecture, implementation, review, verification, release, and process optimization may be executed by different models, providers, runtimes, or physical services. Existing approaches address important parts of this problem: V-model methods define reciprocal relationships between definition and verification; Automotive SPICE formalizes process discipline, traceability, project resources, measurement, and improvement; arc42 provides a practical architecture-document schema; iSAQB AGENTA addresses architecture in agentic software-engineering contexts; Spec-Driven Development and agent harness practices externalize durable specification and repository knowledge; and A2A/MCP provide emerging interoperability mechanisms. However, these elements do not by themselves define an architecture-centered process for assigning heterogeneous AI resources to engineering roles under explicit quality, context, cost, latency, and energy constraints.

This position paper proposes **pArc**, a process for agent-facing engineering in which a quality-gated Architecture Contract is the durable handoff and orchestration substrate. Roles are defined before agents are selected. Each role has responsibility, authority, input/output artifacts, competency requirements, qualification benchmarks, context requirements, and resource constraints. Candidate models and runtimes can then be allocated, measured, replaced, scaled, or removed using cost-performance evidence. The paper intentionally remains a methodology draft: the right-hand verification processes, quantitative thresholds, and empirical validation are research items rather than claimed results.

Keywords: agentic engineering, AI orchestration, software architecture, V-model, architecture contract, role profile, RASIC, Automotive SPICE, arc42, AGENTA, cost-performance optimization, multi-agent systems.

1 Motivation and Problem Statement

Current coding agents are often optimized as vertically integrated products. A planning conversation, implementation tool, execution environment, and context store may belong to one vendor or one session. That is convenient, but it creates an engineering weakness: correctness can depend on ephemeral conversational history or proprietary orchestration state. In long-lived projects, the planner, implementer, reviewer, and verifier may be different services. In enterprise deployments they may even run as separately scaled server pools with different compute economics.

pArc starts from a deliberately different assumption: **agent replacement and role redistribution are normal operating conditions**. The next engineering role must be able to proceed without access to the previous agent’s private chain of conversation. The durable transfer mechanism must therefore be an explicit, versioned engineering artifact.

The process is architecture-centered because architecture is where intent, constraints, decomposition, interfaces, quality attributes, deployment topology, decisions, and verification obligations are made precise enough to support downstream execution. A strong coding model cannot reliably compensate for a missing or ambiguous architecture contract without implicitly repeating architectural work. This motivates an **Architecture Sufficiency Principle**: downstream execution is allowed only after the architecture artifact passes an independent quality gate appropriate to the project.

The second motivation is economic. Human projects already distinguish architects, senior developers, test engineers, project managers, and release roles because required competencies and costs differ. Automotive SPICE MAN.3 likewise treats budget, people, infrastructure, skills, knowledge, experience, and resource allocation as explicit project-management concerns. Agentic engineering should not discard this reasoning. Instead, it can make it more measurable: model benchmark results, effective context, task latency, token use, monetary cost, GPU energy, failure rate, rework rate, and throughput can be observed directly and used for role allocation.

2 Positioning Against Existing Work

Table 1 summarizes the intended relationship. **pArc** does not claim to replace these references; it composes and extends their strongest ideas.

Table 1: Conceptual mapping of major references to pArc.

Reference	Primary contribution	Use in pArc
V-model	Reciprocal definition–verification structure	Symmetric lifecycle and traceability backbone
Automotive SPICE 4.x	Process discipline, resource management, configuration management, measurement, process improvement	Process naming inspiration; MAN.3/SUP.8 concepts; quantitative control and improvement practices
arc42	Tailorable 12-section architecture template	Baseline schema for the Architecture Contract
C4 model	Context, container, component, code and deployment views	View semantics for system and orchestration topology; rendering remains text-based
iSAQB AGENTA 2026.1-rev5	Agent-facing architecture, architectural knowledge provision, alignment, extraction, governance, orchestration concerns	Direct related work for agentic architecture and context governance
Spec-Driven Development / Spec Kit	Durable specification artifacts, clarification, planning, consistency checks, convergence	Iterative refinement and artifact-quality concepts, without adopting vendor/tool-specific scaffolding
OpenAI harness engineering	Repository-local authoritative knowledge; compact entry-point instructions; long-horizon agent work	Session-independent handoff and discoverable engineering knowledge
Agentic Agile-V / SCOPE-V	Agile-V macro-cycle and task-level Specify–Constrain–Orchestrate–Prove–Evolve–Verify loop	Related work for fast agentic iteration with verification discipline
SDAD	Specification, agentic synthesis, independent multi-agent verification, human sign-off	Related work for independent verification and human authority
MCP / A2A	Agent–resource/tool interoperability / agent–agent interoperability	Candidate transport mechanisms; not the engineering methodology itself

The newest direct architecture reference is iSAQB AGENTA, published as an Advanced Level curriculum in 2026. It includes AI-assisted architectural decision-making, providing architectural knowledge to agents, alignment with architectural goals, architecture-information extraction, governance and quality gates, and the evolving architectural role. Its purpose is educational rather than a sequential process-assessment model. **pArc** therefore treats AGENTA as a strong knowledge source while defining an explicit artifact lifecycle, role topology, baseline transition, and resource-allocation mechanism.

3 Core Principles

The current draft proposes ten normative principles. They are intentionally short enough to become the philosophical core of a future public AGENTS.MD charter.

- P1. Agent-Facing Primacy.** Normative engineering information is optimized first for reliable agent consumption. Human-facing pages, PDFs, or wikis are presentation and review projections of the same authoritative artifacts.
- P2. Symmetric Verification.** Every material obligation defined on the left side of the V-model should have an identifiable verification or evidence counterpart on the right side.
- P3. Artifact-Mediated Independence.** Correctness must not depend on one model, provider, session, runtime, or proprietary conversational memory.
- P4. Architecture Sufficiency.** A quality-gated Architecture Contract is a prerequisite for downstream implementation at the claimed level of autonomy.
- P5. Role-Orchestrated Execution.** Engineering roles, responsibilities, authority, inputs, outputs, and competency requirements are defined before selecting agents or models.
- P6. Independent Quality Assurance.** The creator of a normative artifact is not its sole final approver; review must use an independently scoped context and may use a different model/provider when risk justifies it.
- P7. Capability-Proportional Elastic Deployment.** Roles use the lowest-cost resources that satisfy measured capability and quality requirements, while allowing immediate scale-up or model substitution when workload or risk increases.

- P8. Complete Knowledge, Bounded Work Context.** The authoritative knowledge base pursues completeness; individual work packages and context scopes are divided according to agent capability and effective context.
- P9. Quantified Resource Economics.** Quality, accepted output, rework, tokens, monetary cost, energy, latency, throughput, and coordination overhead are measured together.
- P10. Recursive Baseline Convergence.** Fast interactive work remains provisional until it converges into an explicit versioned baseline.

4 Artifact Model

The minimal proposed file structure deliberately avoids tool-specific metadata and document proliferation:

```
Project Root/
  AGENTS.MD          public pArc charter/method
  ARCH.MD            public Architecture Contract
  ARCH_QGate_Template.MD reusable QGate worksheet + user guide
  ARCH_QGate.MD      project-specific independent review evidence
  refs/              text, schema, image, video, PDF, URL
  _pArc/
    SWE1.MD          intent, requirements, constraints
    SWE2.MD          architecture interaction/result ledger
    SWE3.MD          pending, failed, deferred implementation
```

ARCH.MD is the single normative architecture artifact. JSON, YAML, images, videos, PDFs, URLs, or other files are references rather than parallel architecture definitions. Small machine-readable structures may be embedded as fenced blocks. Mermaid is preferred as the default diagram language because the source remains readable by agents while rendering to a human visual in common Markdown environments.

_pArc/SWE2.MD is not the architecture itself. It is the interactive design/process ledger. Each version is represented by a level-2 heading (for example, **## X.Y.Z**) and individual interactions or tasks by level-3 headings. Useful decisions are distilled into ARCH.MD, while failed paths, discussion noise, and transient reasoning need not pollute the Architecture Contract. _pArc/SWE3.MD centralizes deferred, failed, or future implementation work so that unresolved work is not scattered across conversations.

5 Architecture Contract Schema

The proposed ARCH.MD schema begins with the twelve standard arc42 sections, preserving their recognized structure rather than inventing an entirely new architecture document. The pArc draft adds an explicit thirteenth view.

Table 2: Proposed pArc Architecture Contract structure.

No.	Section	pArc treatment
1	Introduction and Goals	Intent, stakeholders, goals, non-goals
2	Architecture Constraints	Product, project, regulatory, tool/runtime constraints
3	Context and Scope	System/environment boundaries; C4 System Context where useful
4	Solution Strategy	Architectural approach and principal trade-offs
5	Building Block View	Static decomposition; C4 Container/Component concepts where useful
6	Runtime View	Dynamic behavior and interactions
7	Deployment View	Runtime topology, infrastructure, network, compute, environments
8	Cross-cutting Concepts	Security, observability, persistence, error handling, common policies
9	Architecture Decisions	ADR-style decisions kept inside ARCH.MD
10	Quality Requirements	Measurable scenarios, acceptance obligations, limits
11	Risks and Technical Debt	Known uncertainty, debt, revisitation triggers
12	Glossary	Domain and process vocabulary
13	Agent Role & Orchestration View	Role/RASIC, capability, artifact I/O, authority, qualification, context, model/runtime placement, cost/energy, scaling and optimization

Section 13 cross-references the arc42 deployment view for physical placement, quality requirements for performance/cost constraints, cross-cutting concepts for common orchestration policies, and architecture decisions for placement rationale. This avoids weakening the original arc42 schema while exposing an engineering concern that does not have a natural dedicated chapter in the original template.

6 Symmetric Lifecycle and the SWE2 Inner Loop

Figure 1 depicts the intended macro-structure. The right side is deliberately shown as provisional in this draft; pArc does not yet prescribe separate SWE5/SWE6 documents. The important requirement is symmetry: right-side evidence must trace to left-side obligations rather than inventing new acceptance semantics during test execution.

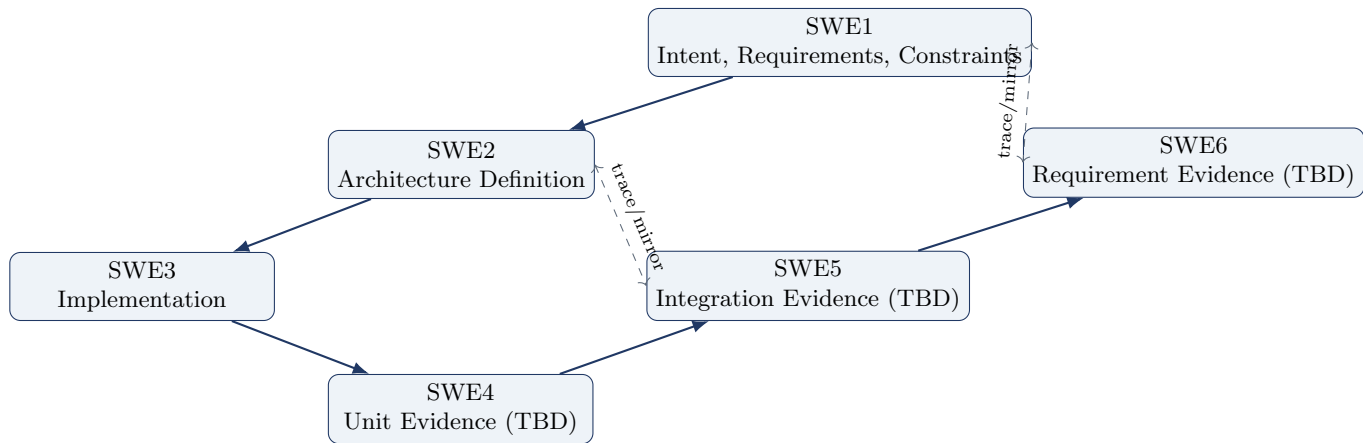


Figure 1: pArc macro V-model: left-side obligations and right-side evidence are reciprocal.

SWE2 contains its own definition/review loop. A design agent produces or updates ARCH.MD. An independently scoped peer role reviews only the authoritative inputs, draft architecture, references, and QGate template rather than inheriting the entire persuasive history of the design conversation. A failed gate returns to SWE2; a passed gate permits baseline transition.

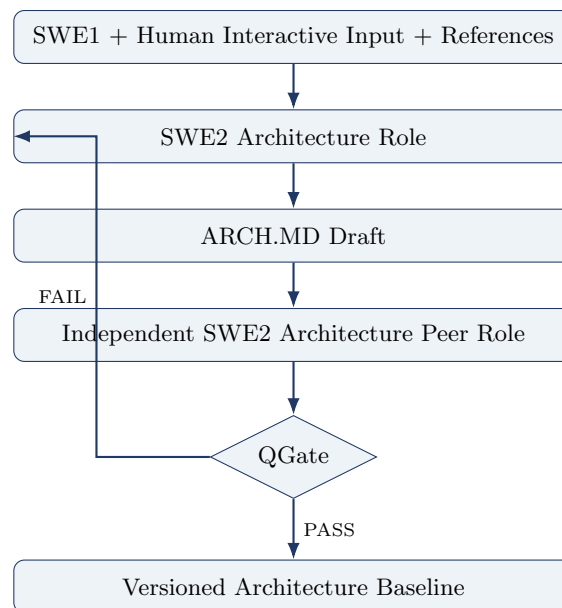


Figure 2: SWE2 inner loop: creation and independent architecture review are separate roles.

7 Role and Responsibility Model

7.1 Process-grouped RASIC

The RASIC matrix should follow engineering/process categories rather than a flat activity list. Table 3 is a draft; it intentionally uses recognizable ASPICE process terms where they fit instead of introducing names such as “Ops AI.” The peer-review role remains a pArc specialization inside SWE2.

Table 3: Draft process-grouped RASIC for major pArc roles. A/R/S/C/I = Accountable/Responsible/Support/Consulted/Informed.

Process	Activity	Human	MAN.3	SWE2 Arch	SWE2 Peer	SWE3	Verify
SUP.8	Baseline/configuration rules	A	R	C	I	I	I
SUP.8	Backup / recovery / config status	A	R	I	I	I	C
MAN.3	Role/profile definition	A	R	C	C	C	C
MAN.3	Budget/resource/model allocation	A	R	C	C	C	C
SWE1	Intent and project constraints	A/R	S	C	C	I	C
SWE1	Acceptance obligations	A	C	R	C	I	C
SWE2	Architecture Contract creation	A	C	R	C	I	I
SWE2	Architecture decisions/deployment view	A	C	R	C	I	C
SWE2	Independent ARCH QGate	A	I	C	R	I	C
SWE3	Implementation to ARCH	I	A	C	I	R	C
SWE4/5/6	Evidence execution/reporting	I	A	C	I	C	R
SPL.2	Product release/package/delivery	A	R	C	I	S	C
PIM.3	Process improvement proposal	A	R	C	C	C	C

SUP.10 is *Change Request Management*, not operations. PIM.3 is *Process Improvement*, not deployment. For production release and delivery, SPL.2 Product Release is the closer Automotive SPICE term. Runtime operations beyond product release are not a strong focus of Automotive SPICE and may require future mapping to IT service/operations references rather than forcing an incorrect ASPICE label.

7.2 Competency, Qualification and Authority

Table 4: Draft AI role profile. Thresholds are project-calibrated rather than hard-coded by pArc.

Role	Competency / skill	Candidate qualification evidence	Authority boundary
MAN.3 Project Management / Orchestration	Work partitioning, scheduling, resource economics, routing, risk-aware scaling	Agentic-workflow benchmark(s), internal orchestration simulation, measured SLA/cost/rework	May allocate/replace resources within approved policy and budget; may not redefine product obligations
SWE2 Architecture	Requirements analysis, architecture synthesis, trade-offs, C4/arc42 views, ADR, quality and deployment	ArchBench; independent general-reasoning index; internal ARCH corpus/QGate eval	May draft architecture and propose decisions; cannot self-approve baseline
SWE2 Architecture Peer	Omission/contradiction/risk detection, architecture quality assessment	Same architecture portfolio with stricter project threshold plus independent-review miss-rate	May PASS/FAIL QGate; final business authority remains human
SWE3 Implementation	Repository navigation, coding, debugging, tool use, conformance to ARCH	Artificial Analysis Coding Agent Index v1.5; SWE-bench family with caveats; internal repository acceptance suite	May modify source within ARCH; silent architecture change prohibited
SWE4/5/6 Verification	Deterministic execution, evidence collection, trace reporting; higher reasoning only when analysis is required	Test reproducibility, false-pass/false-fail, coverage, tool benchmark; role-specific reasoning benchmark if analytical verification is needed	May report evidence/pass-fail against predefined criteria; cannot invent acceptance semantics
SPL.2 Product Release	Packaging, release configuration, delivery checks	Deployment/release rehearsal, rollback success, package integrity metrics	May execute approved release/rollback policy
PIM.3 Process Improvement	Measurement interpretation, optimization proposals	Historical cost/quality improvement, policy simulation	May recommend changes; governance/human authority accepts process changes

8 Benchmark Portfolio Rather Than a Single Score

SWE-bench measures repository-level issue resolution: a model receives a codebase and GitHub issue and must produce a patch that passes tests. Reasoning is therefore used implicitly but is not isolated as a pure reasoning metric. It is appropriate evidence for SWE3 coding capability, but not sufficient for architecture or orchestration roles.

For SWE2, ArchBench is a more direct reference because it was introduced specifically to benchmark generative AI on software-architecture tasks. Architecture qualification should also include architecture knowledge and project-specific internal evaluations. For general reasoning, candidate external signals include Humanity’s Last Exam, ARC-AGI-2, and independently run composite indices such as the Artificial Analysis Intelligence Index. The latter is especially relevant to pArc because it reports score together with token usage, cost per task, and other performance measures, allowing cost-performance comparisons rather than capability-only ranking.

As of July 2026, there is no single OpenAI-designated replacement for SWE-bench Pro. OpenAI first recommended Pro after finding contamination/design issues in SWE-bench Verified, but later audited SWE-bench Pro, estimated roughly 30% of tasks to be broken, and explicitly retracted the recommendation. OpenAI instead called for new benchmarks built by experienced software developers with strong human oversight. Contemporary OpenAI model reports therefore use a portfolio of coding and agentic evaluations rather than one successor metric, including independent coding-agent indices and terminal/long-horizon evaluations. This supports the pArc position that qualification is role-specific and multi-metric.

Table 5: Candidate benchmark sources by pArc role. This table names evidence sources, not fixed thresholds.

Role concern	Candidate benchmark	Interpretation
Software architecture	ArchBench; independent reasoning index; internal ARCH/QGate eval	Architecture task/knowledge capability and project-specific conformance
Repository coding	Artificial Analysis Coding Agent Index v1.5; SWE-bench ecosystem (with caveats); internal acceptance suite	Issue resolution, terminal/tool execution, actual repository success
General reasoning	HLE; ARC-AGI-2; independent intelligence composite	Broad reasoning signal, not a substitute for role-specific architecture/coding evaluation
Long-context work	Independent long-context reasoning evals; provider data as initial declaration	Context effectiveness; pArc primarily relies on measured project outcomes after deployment
Tool/agent workflows	Terminal-Bench / automation-agent evaluations / internal workflow simulation	Long-horizon execution and tool use
Cost efficiency	Independent cost-per-task data + project telemetry	Capability must be paired with monetary/token/latency/energy metrics

9 Quantitative Resource Economics

The pArc orchestrator should behave more like a project-resource manager and cloud scheduler than a simple model router. A cheap model is not economical if it creates excessive rework, handoffs, or integration failures. Conversely, a high-cost architecture peer may be economical if it is invoked only at architecture baseline transitions.

For an accepted work unit, a useful first-order cost is

$$C_{accepted} = \frac{C_{inference} + C_{tools} + C_{compute} + C_{energy} + C_{review} + C_{rework} + C_{coord}}{N_{accepted}}. \quad (1)$$

The corresponding performance vector should be retained with the cost rather than reporting monetary cost alone:

$$P = (Q, R, L, T, E, C), \quad (2)$$

where Q is accepted quality/conformance, R rework or failure rate, L latency, T throughput, E energy, and C monetary/token cost. This mirrors capacity-planning practice: deployment is optimized from measured utilization and service-level behavior after an initial run.

Work partitioning can be formalized without pretending that one universal constant exists. For a work package w and candidate agent a ,

$$Context(w) \leq \alpha_{a,p} EffectiveContext(a, p), \quad (3)$$

$$Capability(a, p) \geq Requirement(w, p), \quad (4)$$

where p identifies the project/role profile and α is calibrated empirically. Among feasible candidates, orchestration minimizes an objective such as

$$J = C + \lambda_1 R + \lambda_2 L + \lambda_3 E + \lambda_4 H, \quad (5)$$

where H represents coordination/handoff overhead. The weights are project policy, not universal pArc constants.

This model captures the same trade-off seen in staffing: many inexpensive agents may require smaller tasks and more coordination, whereas fewer expensive agents may absorb larger context and reduce handoff/rework. The economically relevant question is therefore not “which model is cheapest?” but “which role allocation minimizes total accepted-output cost at the required quality and risk level?”

10 Architecture-Centric Orchestration and Deployment

The architecture contract is both a product-design artifact and an orchestration boundary. A pArc deployment can evolve from a single interactive environment to physically separated services without changing the engineering contract. Figure 3 shows the logical role topology.

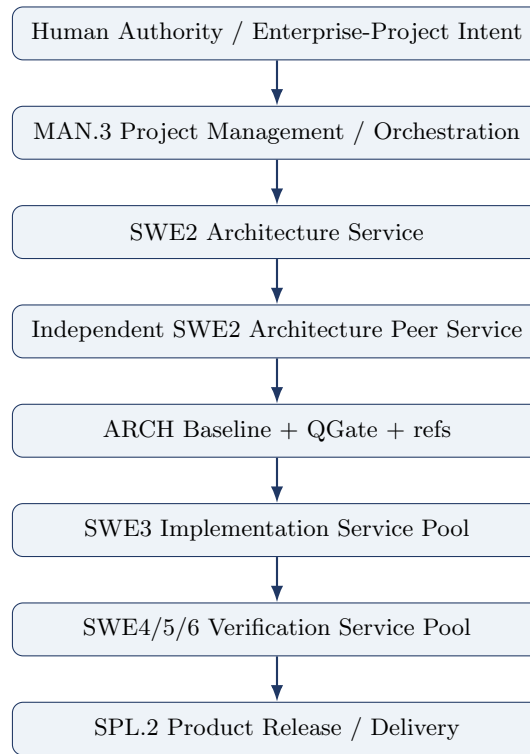


Figure 3: Role topology: services may be different models, providers, runtimes, or physical server pools.

This is deliberately not tied to A2A, MCP, GitHub, GitLab, or any other orchestration runtime. MCP may expose artifacts and tools; A2A may support inter-agent task delegation; a proprietary platform may provide routing. The normative pArc layer is above these mechanisms: role profiles and artifact contracts survive a runtime change.

The deployment view should include scaling policy. For example, an expensive architecture peer is mandatory while the architecture changes materially, but can be removed from the normal path when the architecture is stable and implementation changes remain within predefined boundaries. Repeated SWE3 failures, an architecture-sensitive module change, or a quality threshold breach can trigger peer reactivation. Similarly, implementation capacity can scale horizontally while keeping architecture authority centralized.

11 Quality Gate and Verification Philosophy

`ARCH_QGate_Template.MD` combines the copy-ready evaluation worksheet with its definition and user guide. It uses the arc42 chapter schema as its primary structural baseline, then overlays process/quality criteria from ASPICE, AGENTA, SDD/harness practices, and pArc-specific orchestration concerns. Candidate checks include completeness, consistency, ambiguity, traceability, testability, explicit N/A/deferred status, architecture-deployment consistency, role/authority completeness, context suitability, independent-review separation, and cost/performance observability.

Safety-critical or business-critical acceptance criteria belong on the left side of the V. The right side is a mirror that executes tests, analyses, simulations, measurements, or inspections and produces evidence against those already-defined criteria. Verification may design instrumentation or execution mechanics, but it should not silently invent a new meaning of “acceptable.” This is especially important in agentic systems because otherwise a downstream verifier can unintentionally redefine the target while attempting to validate it.

ASPICE capability-level labels are not adopted as pArc maturity grades. However, the substance behind quantitative analysis/control and process innovation is useful. Instead of stopping at an audit level, pArc places measurable

resource economics and closed-loop optimization directly into the expected operating model and QGate concerns when applicable.

12 Research Questions and Evaluation Plan

The present work is a position/methodology proposal. Evidence is needed before claiming superiority. The following research questions are feasible with real projects:

- RQ1.** Does an Architecture Contract allow a different implementation agent/provider to continue work with lower architecture drift than session-history handoff?
- RQ2.** Does independent SWE2 peer review detect omissions/contradictions that self-review misses, and what is the marginal cost per defect found?
- RQ3.** Can role-specific model allocation reduce total accepted-output cost compared with using one frontier model for all stages?
- RQ4.** How should work-package size vary with effective context, benchmark capability, and coordination cost?
- RQ5.** Can architecture freeze reduce recurring high-cost peer-review capacity without increasing integration failures?
- RQ6.** Do token, monetary cost, energy, latency, and rework jointly form a useful placement objective for enterprise agent infrastructure?

A practical empirical program would use two or three open-source or personal projects of different sizes. Each project would maintain the same pArc artifacts, then run controlled substitutions of architecture, implementation, and review models. Measured outputs would include QGate findings, architectural conformance defects, implementation success, regression failure, context/token use, wall-clock latency, cost, energy where observable, and rework. Static public leaderboards are useful for initial qualification but project telemetry should dominate later placement decisions.

13 Threats to Validity and Open Issues

Several limitations are explicit. First, public benchmark quality is unstable. SWE-bench Verified suffered contamination, and SWE-bench Pro was later found to contain substantial task defects. This motivates a portfolio and internal calibration but also makes cross-provider comparison difficult. Second, energy measurement is often unavailable for hosted inference and may need provider reporting or estimation. Third, independently scoped review using the same model family is not fully independent; heterogeneous model/provider review may reduce correlated failure but costs more. Fourth, the right-hand V-model and release/operations semantics are not yet fully specified in pArc. Fifth, the proposed role profiles and optimization objective require empirical calibration rather than arbitrary universal thresholds.

Finally, **pArc** should not be presented as an Automotive SPICE derivative or compliance model. It borrows process concepts and numbering conventions where useful while targeting a different purpose: operational orchestration of heterogeneous AI engineering resources around durable architecture artifacts.

14 Conclusion

pArc proposes an architecture-centric answer to a practical enterprise question: if AI agents become engineering resources rather than one assistant session, how should roles, artifacts, quality gates, and compute economics be organized? The core answer is to make the Architecture Contract the durable handoff substrate, define roles before selecting agents, mirror left-side obligations with right-side evidence, qualify agents using role-specific benchmark portfolios, and continuously optimize deployment using measured quality and cost-performance data.

The proposal is intentionally incomplete. Its immediate next work is to finalize the arc42-based Architecture Contract schema, validate and refine the Architecture QGate template, define formal role profiles and baseline transitions, and then test the approach across real projects with agent substitution and cost/rework measurement. If those experiments support the hypotheses, pArc could mature from a position paper into a reusable open engineering method and a concrete contribution to the emerging agentic architecture community.

References

- [1] arc42. *arc42 Documentation / Overview*. <https://arc42.org/overview/>
- [2] VDA QMC. *Automotive SPICE Process Assessment Model*, current 4.x family and 2026 update context. <https://vda-qmc.de/en/automotive-spice/>
- [3] iSAQB. *AGENTA – Architecture for Agentic Software Engineering Contexts*, 2026.1-rev5. <https://public.isaqb.org/curriculum-agenta/curriculum-agenta-en.pdf>
- [4] Simon Brown. *The C4 Model for Visualising Software Architecture*. <https://c4model.com/>
- [5] GitHub. *Spec Kit / Spec-Driven Development*. <https://github.com/github/spec-kit>
- [6] OpenAI. *Harness Engineering: Leveraging Codex in an Agent-First World*. <https://openai.com/index/harness-engineering/>
- [7] OpenAI. *Separating Signal from Noise in Coding Evaluations*, 8 July 2026. <https://openai.com/index/separating-signal-from-noise-coding-evaluations/>
- [8] SWE-bench. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* <https://www.swebench.com/>
- [9] ICSA 2026. *ArchBench: Benchmarking Generative-AI for Software Architecture Tasks*. <https://arxiv.org/abs/2603.17833>
- [10] Artificial Analysis. *Coding Agent Index v1.5*, combining DeepSWE v1.1, Terminal-Bench 4.0, and SWE-Atlas-QnA with cost/token/time metrics. <https://artificialanalysis.ai/agents/coding-agents>
- [11] Terminal-Bench. *Terminal-Bench 4.0*. <https://www.tbench.ai/>
- [12] Artificial Analysis. *Artificial Analysis Intelligence Index v4.3*. <https://artificialanalysis.ai/evaluations/artificial-analysis-intelligence-index>
- [13] ARC Prize. *ARC-AGI-2*. <https://arcprize.org/arc-agi/2>
- [14] A2A Protocol. *Agent2Agent Protocol Specification*. <https://a2a-protocol.org/>
- [15] Model Context Protocol. *MCP Specification*. <https://modelcontextprotocol.io/>
- [16] *Agentic Agile-V / SCOPE-V*, arXiv:2605.20456 (2026).
- [17] *Specification-Driven Agentic Development (SDAD)*, arXiv:2608.20341 (2026).